Industrial Experience Report

Kevin Worner

April 23, 2013

CSC 465/467

Innovative Systems LLC: May 2011 - Present

I have had the good fortune of working as an intern for Innovative Systems LLC for two years and the opportunity to work on many projects with them. Last summer, I took coop credit for the summer internship.

As an intern, I worked with a small group working closely with the main office to build mobile applications, internal testing tools, and most recently I worked with 2 others on our senior design project for school that was hosted by Innovative Systems.

Appendix 1 – Coop report

About

My internship this summer with Innovative Systems was fun, creative, and full of learning experiences. I learned more about my personal goals, motivations, and pet peeves. I had the opportunity to learn and use a different language and to develop on an extremely popular platform. But one of the best parts of the summer is the feeling that I contributed to the success of a company, not by buying their products but instead by designing and creating one of them.

This summer I worked with Michael Malkowski and Brian Butterfield to design and create a visual voicemail application for the Android mobile device platform. A simplistic description of what this encompasses: The receiving and managing of your landline voicemails, on your cellphone. Using Innovative Systems' APIs to access voicemails from an account, we were able to download information, audio, and the transcription of each voicemail. Information included who the voicemail was from, when it was recorded, how long it is, whether the user has played the audio before, and other small bits of information that facilitated audio transfer. The transcription was a text-string that was provided by a third-party service. The audio was an ADPCM wave file.

The rest of this report is an explanation of each of the components of the app.

Communication

Asynchronous Pattern

It turns out that Android doesn't have built-in asynchronous networking, unlike the Windows Phone or the iPhone. There are many third party APIs, but nothing from Android. Not wanting to use any third party code, we decided to find a different way to do it. Android does have threads, and it has a template class called AsyncTask.

AsyncTask is a wrapper for threads that does some work, and then joins back onto the main thread. After a quick proof of concept, I put the synchronous networking code in this wrapper to effectively get asynchronous networking.

I had to write a class for each type of message we wanted to send. Each class implemented AsyncTask and looked pretty similar, as there are 4 steps that get overridden.

1) onPreExecute() – this is called on the UI thread right after the task is told to start. This is where you can do any setup needed for the rest of the task. I used this step to setup the communications wrapper (see below).

2) doInBackground(params …) – this is called on the background thread as soon as step 1 is complete. It takes an array of parameters, whose type is declared with the class.

Eg:

```
public class MyTask extends AsyncTask<ParamType, ProgressType, ReturnType>
```

It is in this step that I put the synchronous network call.

3) onProgressUpdate(progress …) – this step is called on the UI thread anytime publishProgress(progress …) is called from the second step, and is usually used to update a progress bar or post some log messages. I didn't use this step.

4) onPostExecute(result) – this is called on the UI thread after step 2 is complete. Whatever is returned from doInBackground() is passed to here as a parameter. I decided to call another function, in my main android class, to handle the return message since each message type can be used to send/receive different messages. For example: pbMailboxCommand can be used to update messages, delete messages, or mark messages as deleted. And each of these messages has a different object that is returned.

Protocol Buffers

We used a platform built by Google to serialize our data structures for network communication. This platform is called Protocol Buffers and works similar to XML, but is much smaller. First you define your structure in a .proto file, which you then compile. This compiled file has all the serialization methods in it so all you need to do is create one of your objects, set the desired members, and call .build(). You can then pass the resulting object into the communication wrapper for it to be serialized and sent.

Wrapper

We used a custom wrapper built around TCP to manage our communications. It would take a message object, insert CRC32 checksums, encrypt it, and then send it. For encryption we used AES 128 which is built into java's security libraries.

A CRC, or CRC32, is a 32 bit checksum that is used to determine whether or not any data was corrupted when it was transmitted. From accuhash.com: "*Cyclic redundancy checksum (CRC) is the remainder of a binary division with no bit carry (XOR used instead of subtraction), of the message bit stream, by a predefined (short) bit stream. In short, take the data as a VERY long binary number and divide it by a constant divisor.*"

When we received a response from the server, the wrapper reversed its process. It decrypted the message, checked the CRC's and removed them, then returned a byte array. We passed this

byte array into its corresponding protocol buffer object and created a new object that we could work with.

Data/Messages

Below are some of the messages that were used in the app, and a brief description of them.

Voicemails and Refresh

A major part of this app was to be able to look at and listen to your land line voicemails. To do that, we had to first download them to our phone. We had two main messages for that: RefreshAllMessages() and RefreshMessagesNewerThan(int time).

They do exactly as they sound. The first message asks the server for every message in the account we are logged in to. The second message asks the server for only messages that are newer than a given time. The time is expressed as the number of seconds since 12:00am January 1st, 1970. This number fits into an unsigned 32 bit integer.

Some of the data that can be returned for each message are:

- Transcription
- Audio
- Audio Length
- Sender
- Sender medium
- File location on the server
- Priority of the voicemail (normal or urgent)

RefreshAllMessages was only called when a user first logged into the app, and then stored its update time in the internal sqlite database. RefreshMessagesNewerThan was called when a user reopened the app, or when a user pushed the refresh button.

Because the voicemails are available to manage from other sources than our app, we have to synchronize with the server. To do this, I get a list of voicemail IDs from the server and a list from our app. I delete from our app any voicemails that are not in the list from the server, this updates any voicemails that may have been deleted elsewhere. Then I add any voicemails that exist on the server but not in the app. This updates any new voicemails.

Login

To be able to access their messages, users had to login to our app which also logged them into the servers. Users had to provide Username, Password, Mailbox Pin, and Server URL. The username and password got them into their account, the mailbox pin is an additional password needed to access and modify the voicemails, and the server URL is an outward facing (accessible from the internet) access point to the APMax server.

When a user pressed login, a chain of four steps was started. If any of the first three failed, the login failed. First, the username, password, and pin were validated on the server. Second, the app was reconfigured so that communication was sent to a specific processor on the APMax, rather than the admin processor. Third, the app registered for a Google Push Notification that informed users when they received a new voicemail. And finally fourth, all voicemails were downloaded. If the voicemails failed to download, a user could manually refresh the list.

Once a user is logged in, they stay logged in until they explicitly log out.

Logout

If a user wanted, they could log out. This would bring them back to the login page and a different user could login on the same phone.

The logout command did two things. It informed the server of the logout request, which returned a simple acknowledgment, and it cleared the database of all its data. This returned the app to its default communication configuration as well.

Mark as Read/Delete

Some of the actions you could apply to a message were to mark it as read, or mark it for deletion.

When you played a message, it was automatically marked as read, but you could also mark it as read without having to listen to it.

When you no longer wanted the voicemail anymore, you could delete it. This not only deleted the message from the phone's database, but also sent a message to the server to delete it there as well.

Database

The Android platform comes with a Sqlite database built into it for use by the operating system as well as any applications. We used it to store two tables, one for the user and their information, and one for the messages.

Account

This table holds the username, hashed password, pin, URL, last time of update, the Google push notification id, a signed in flag, and several other ID's used in the communications. There is only ever one row in this table because there is only ever one person logged in at a time.

Message

Each row in this table holds a voicemail message. The audio is stored as a blob and is not required to be present. Other data pieces are the download time, audio length, where the audio is stored on the server (relative path), the audio transcription, the phone number that left the

message, the date the message was left, whether the message has been listened to yet, and whether the audio has been downloaded yet.

GUI

I had very little work dealing with the user interface of the app. This portion was almost entirely done by Mike.

Playing Audio

We had a difficult time getting audio to play. The audio on the server is stored as an ADPCM wave file, and the Android device could only play PCM type wave files (in addition to mp3s). The difference between the two types is that a PCM has all the sampling data, whereas the ADPCM has the DIFFERENCE between the samples. ADPCM saves quite a bit of space since it doesn't have to store all of the data, only how the data has changed.

To get the audio into PCM format, we had to use a file converter that the main office in Mitchell had written. This added another dependency that we hadn't planned originally planned on.

Testing

Testing was also a large part of this app. We wanted to have unit tests backing all major components so we could be confident that everything was working and that when we changed something we could be confident that nothing broke. We used the test platform, JUnit, which was built into Android.

The idea was to write our tests as we went so that they would be built up over time and we could use them while writing new code. It didn't go as planned. In the end, we ended up writing all our tests after we were done and this is a terrible way to write tests! After my experience this summer, I am much more apt to either write my test cases first or meticulously plan the method prototypes before writing any implementation code.

Because I was writing the tests after writing the code, I found my code to be very test-unfriendly and as a result started re-writing some of the methods so that they could be tested more efficiently.

Other Features

Callback

One of the extra features we added to the project was the ability to call a number that left a message. You pick the message, hit the callback button, and phone will start calling that number.

The implementation of this feature was actually quite simple. All that needs to be done is retrieve the phone number from the message, and pass it to the Android operating system with the ACTION_CALL command. The OS will then take over, place the call, and when the call is complete it will return control to the application.

Log

For debugging purposes, we added a custom logger to the app. We defined different log levels (error, info, landmark, other) and logged important info throughout the app. Anytime something significant happened, a log entry was made. This log was available to view by the user but was meant only for debugging, so this feature will be available only to an admin in a future version.

Things you could do with the log: view it on the phone, email it to another person (a customer service representative for instance), and set the level at which entries are logged. The higher the log level, the more log entries. These entries were kept in their own database.

Context menus

For convenience, we also added short-cut context menus. If you were to tap and hold on a message, a menu would come up that would let you mark the message as read, delete it, or call back the person who left it. This lets you get to the extra options without having to go to the extra step of opening the voicemail.